

AI, LLM, and Model Routing

Providers, model routing, fallback, policy, and cost accounting.

Eurskem AI · Agentic Workflow Platform

Generated 2026-08-24

Code revision 8bd16ca4a2a3

Derived from the live repository

Source of truth. Inventories and symbol tables are generated from the current checkout. Narrative explains observed structures and does not replace executable contracts.

Model access architecture

Nodes request models through the shared LLM gateway. The registry resolves aliases and provider metadata; policy selects eligible models; contextual clones bind run, session, and node IDs for cost attribution. Retry and fallback record intended and actual models.

- Model identifiers are registry validated.
- Provider calls use finite timeouts and bounded retries.
- Usage is recorded at run/node boundaries.
- Policy-sensitive decisions fail closed.
- Structured output is parsed and validated.

File	Responsibility
<code>app/llm/__init__.py</code>	LLM gateway package.
<code>app/llm/anthropic_batch.py</code>	Anthropic Message Batches API -- standalone, opt-in batch service.
<code>app/llm/anthropic_gw.py</code>	Anthropic Claude gateway.
<code>app/llm/base.py</code>	Abstract LLM gateway contract.
<code>app/llm/batch_types.py</code>	Provider-neutral shapes for the Anthropic/OpenAI Batch API services.
<code>app/llm/catalog.py</code>	Approved model catalog shared by the runtime, API, and Workflow Builder.
<code>app/llm/errors.py</code>	Provider-neutral LLM errors used by the resilience layer.
<code>app/llm/local_openai_gw.py</code>	OpenAI-compatible gateway for privately hosted Kimi K3 and GLM-5.
<code>app/llm/model_catalog.py</code>	Provider-neutral LLM catalog used by routing, validation, cost, and UI.
<code>app/llm/model_router.py</code>	Deterministic, zero-token model selection for automatic workflow routing.
<code>app/llm/openai_batch.py</code>	OpenAI Batch API -- standalone, opt-in batch service.
<code>app/llm/openai_gw.py</code>	OpenAI implementation of LLMGateway.
<code>app/llm/openai_registry.py</code>	Authoritative OpenAI model registry for every platform capability.
<code>app/llm/openrouter_catalog.py</code>	Live OpenRouter model catalog, sourced directly from OpenRouter's own <code>`GET /v1/models`</code> .
<code>app/llm/openrouter_gw.py</code>	OpenRouter implementation of LLMGateway — direct calls to OpenRouter's real API.
<code>app/llm/openrouter_ranking.py</code>	Auto-router config for OpenRouter — see docs/architecture/LLM_MODEL_ROUTING.md.
<code>app/llm/registry.py</code>	Resolve a model name to the right gateway instance.
<code>app/llm/semantic_cache.py</code>	Tenant-scoped Redis semantic cache for deterministic text completions.
<code>app/nodes/_stubs.py</code>	Stub nodes used only to test the runtime in isolation from LLMs/RAG.
<code>app/nodes/ai_task.py</code>	AITaskAgent — one configurable AI capability.
<code>app/nodes/consistency_checker.py</code>	ConsistencyChecker — a DETERMINISTIC node (no LLM) that gates the render.
<code>app/nodes/data_transform.py</code>	DataTransformAgent — deterministic data shaping.

File	Responsibility
<code>app/nodes/decision.py</code>	DecisionAgent — deterministic business logic, no model call.
<code>app/nodes/horizon_evaluation.py</code>	Workflow node for independent two-provider Horizon evaluation.
<code>app/nodes/integration.py</code>	IntegrationAgent — one cloud-storage capability with a provider selector.
<code>app/nodes/mcp_agent.py</code>	MCPAgent: an LLM-driven agent loop calling MCP tools.
<code>app/nodes/router.py</code>	RouterAgent: the workflow's branching primitive.
<code>app/nodes/scientific_skill_agent.py</code>	LLM research node guided by approved K-Dense Scientific Agent Skills.
<code>app/nodes/transform.py</code>	TransformAgent: pure LLM transform with structured output.
<code>app/observability/cost_ledger.py</code>	Cost ledger module.
<code>app/security/llm_policy.py</code>	Data classification + OPA policy enforcement for direct provider calls.